SIMATS ENGINEERING — Assessment Tool 3

Database Management Systems (CSA05) — CO3: Normalization & Functional Dependencies
MCQ Answer Key with Explanations and SQL Demonstrations

Name: P SHASHANK | Reg No: 192524282

Each question below shows the correct option, a plain-language explanation, and a short SQL demonstration (styled as a terminal/CLI screenshot) illustrating the concept using standard SQL syntax compatible with MySQL. Demonstrations were executed in a SQL engine in this sandbox since a live MySQL server isn't installable here (no network access), but the SQL shown is standard DDL/DML that runs the same way in MySQL.

Q1. Which of the following is true about 1NF?

Bloom's Level: BL2 – Understand | Marks: 5

(a) Allows multi-valued attributes

(b) Eliminates partial dependencies

✓ (c) All attributes must be atomic — CORRECT ANSWER

(d) Eliminates transitive dependencies

Explanation:

First Normal Form (1NF) requires every column to hold a single, indivisible (atomic) value — no repeating groups or comma-separated lists in one cell. Eliminating partial dependencies is the job of 2NF, and eliminating transitive dependencies is the job of 3NF, so those describe higher normal forms, not 1NF itself.

```
Q1 Demo: 1NF requires atomic (single-valued) attributes
mysql> SELECT * FROM Student_BadDesign;
+-----------+------+-----------------------+
| StudentID | Name | Phone                 |
+-----------+------+-----------------------+
| 1         | Arun | 9876543210,9123456780 |
+-----------+------+-----------------------+
1 row in set

mysql> SELECT * FROM Student_1NF;
+-----------+------+------------+
| StudentID | Name | Phone      |
+-----------+------+------------+
| 1         | Arun | 9876543210 |
| 1         | Arun | 9123456780 |
+-----------+------+------------+
2 rows in set
```

Q2. A relation is in 2NF if it is in 1NF and:

Bloom's Level: BL2 – Understand | Marks: 5

(a) Has no transitive dependencies

✓ (b) Has no partial dependencies on primary key — CORRECT ANSWER

(c) Every attribute is a prime attribute

(d) Has no multi-valued dependencies

Explanation:

2NF builds on 1NF by additionally requiring that every non-key attribute depend on the WHOLE primary key, not just part of it. A 'partial dependency' happens only when the key is composite (more than one column) and some attribute depends on just one part of that key — as with StudentName depending only on StudentID below, not on the full (StudentID, CourseID) key.

```
Q2 Demo: 2NF removes partial dependency on the primary key
mysql> SELECT * FROM Enroll_BadDesign;  -- StudentName depends only on StudentID (partial dependency)
+-----------+----------+-------------+-------+
| StudentID | CourseID | StudentName | Grade |
+-----------+----------+-------------+-------+
| 1         | 101      | Arun        | A     |
| 1         | 102      | Arun        | B     |
+-----------+----------+-------------+-------+
2 rows in set

mysql> SELECT * FROM Student; -- split out: no partial dependency now
+-----------+-------------+
| StudentID | StudentName |
+-----------+-------------+
| 1         | Arun        |
+-----------+-------------+
1 row in set
```

Q3. Which normal form deals with multi-valued dependencies?

Bloom's Level: BL2 – Understand | Marks: 5

(a) 2NF
(b) 3NF
(c) BCNF

✓ (d) 4NF — CORRECT ANSWER

Explanation:
Fourth Normal Form (4NF) specifically targets multi-valued dependencies (MVDs) — situations where one key is associated with multiple independent sets of values (e.g., an employee's skills and the languages they speak), causing combinatorial redundancy that 2NF/3NF/BCNF do not address.

```
Q3 Demo
mysql> SELECT * FROM Emp_MVD; -- Skill and Language are independent MVDs -> redundancy, violates 4NF
+-------+--------+----------+
| EmpID | Skill  | Language |
+-------+--------+----------+
| 1     | SQL    | English  |
| 1     | SQL    | Tamil    |
| 1     | Python | English  |
| 1     | Python | Tamil    |
+-------+--------+----------+
4 rows in set

mysql> SELECT * FROM Emp_Skill; -- decomposed to 4NF, no redundant combinations
+-------+--------+
| EmpID | Skill  |
+-------+--------+
| 1     | SQL    |
| 1     | Python |
+-------+--------+
2 rows in set
```

Q4. BCNF is a stricter version of:

Bloom's Level: BL2 – Understand | Marks: 5

(a) 1NF
(b) 2NF
✓ **(c) 3NF — CORRECT ANSWER**
(d) 4NF

Explanation:
Boyce-Codd Normal Form (BCNF) tightens 3NF: in 3NF a non-prime attribute may still be non-transitively dependent through a determinant that is not a candidate key, but BCNF requires that EVERY determinant (left-hand side of a functional dependency) be a candidate key. Every relation in BCNF is automatically in 3NF, but not vice versa.

```
Q4 Demo
mysql> SELECT * FROM Course_3NF_not_BCNF; -- Course->Instructor holds, Course is not a candidate key: viola
+-----------+--------+------------+
| StudentID | Course | Instructor |
+-----------+--------+------------+
| 1         | DBMS   | Dr.Kumar   |
| 2         | DBMS   | Dr.Kumar   |
+-----------+--------+------------+
2 rows in set

mysql> SELECT * FROM Course_Instructor; -- every determinant is now a candidate key: satisfies BCNF
+--------+------------+
| Course | Instructor |
+--------+------------+
| DBMS   | Dr.Kumar   |
+--------+------------+
1 row in set
```

Q5. Functional dependency X -> Y means:

Bloom's Level: BL2 – Understand | Marks: 5

(a) Y determines X
✓ **(b) X determines Y — CORRECT ANSWER**
(c) X and Y are unrelated (d)
Both determine each other

Explanation:
X -> Y (read 'X determines Y') means that for every value of X there is exactly one associated value of Y — knowing X tells you Y uniquely. It says nothing about Y determining X; that would be the separate dependency Y -> X.

```
Q5 Demo
mysql> SELECT RollNo, Name FROM FD_Demo ORDER BY RollNo; -- RollNo -> Name: each RollNo determines exactly
+--------+------+
| RollNo | Name |
+--------+------+
| 1      | Arun |
| 2      | Bala |
| 3      | Arun |
+--------+------+
3 rows in set
```

Q6. Which of the following anomaly is NOT associated with unnormalized relations?

Bloom's Level: BL2 – Understand | Marks: 5

(a) Insertion anomaly
(b) Deletion anomaly
(c) Update anomaly

✓ (d) Retrieval anomaly — CORRECT ANSWER

Explanation:
Unnormalized (redundant) tables classically suffer from insertion, deletion, and update anomalies — problems caused by storing the same fact in multiple rows. 'Retrieval anomaly' is not a recognized term in normalization theory; retrieving (SELECTing) data is unaffected by normalization the way modifying it is.

```
Q6 Demo
mysql> SELECT * FROM Unnormalized; -- before delete: 'IT' dept info stored redundantly on every employee ro
+-------+---------+----------+
| EmpID | EmpName | DeptName |
+-------+---------+----------+
| 1     | Arun    | IT       |
| 2     | Bala    | IT       |
+-------+---------+----------+
2 rows in set

mysql> SELECT * FROM Unnormalized; -- after deleting Arun (only remaining row shown): if Bala didn't exist,
+-------+---------+----------+
| EmpID | EmpName | DeptName |
+-------+---------+----------+
| 2     | Bala    | IT       |
+-------+---------+----------+
1 row in set
```

Q7. A superkey is:

Bloom's Level: BL2 – Understand | Marks: 5

(a) A minimal set of attributes that uniquely identifies tuples

✓ (b) Any set of attributes that uniquely identifies tuples — CORRECT ANSWER

(c) An attribute that depends on a non-prime attribute
(d) A foreign key reference

Explanation:
A superkey is ANY combination of attributes whose values uniquely identify a row — it need not be minimal. Once you strip a superkey down so that no attribute can be removed without losing uniqueness, it becomes a candidate key. Option (a) actually describes a candidate key, not a superkey.

```
Q7 Demo
mysql> SELECT DISTINCT RollNo, Email FROM SuperKeyDemo; -- {RollNo,Email} uniquely identifies rows: it's a
+--------+---------+
| RollNo | Email   |
+--------+---------+
| 1      | a@x.com |
| 2      | b@x.com |
+--------+---------+
2 rows in set
```

Q8. If R(A,B,C) has FD: A->B and B->C, then C is:

Bloom's Level: BL2 – Understand | Marks: 5

(a) Partially dependent on A
(b) Fully dependent on A

✓ (c) Transitively dependent on A — CORRECT ANSWER
(d) Independently determined

Explanation:
Because A->B and B->C, we get A->C only indirectly, through the intermediate attribute B. This chain (A determines C only via B, where B is not part of a key) is the textbook definition of a transitive dependency, and it is exactly what 3NF removes.

```
Q8 Demo
mysql> SELECT * FROM Transitive_Demo; -- A->B and B->C, so A->C holds only via B: C is transitively depende
+---+----+----+
| A | B  | C  |
+---+----+----+
| 1 | B1 | C1 |
| 2 | B1 | C1 |
+---+----+----+
2 rows in set
```

Q9. 5NF is also known as:

Bloom's Level: BL2 – Understand | Marks: 5

(a) Domain-Key Normal Form
(b) Boyce-Codd Normal Form

✓ (c) Project-Join Normal Form — CORRECT ANSWER(d) Element Normal Form

Explanation:
Fifth Normal Form (5NF) is also called Project-Join Normal Form (PJNF) because it is defined in terms of join dependencies: a relation is in 5NF when every join dependency in it is implied by its candidate keys, i.e., it cannot be decomposed further without losing information on rejoin.

```
Q9 Demo
mysql> SELECT * FROM Supply; -- join-dependency exists among 3 attributes: decompose losslessly to reach 5N
+--------+----------+----------+
| Agent  | Company  | Product  |
+--------+----------+----------+
| AgentA | CompanyX | Product1 |
| AgentA | CompanyY | Product1 |
+--------+----------+----------+
2 rows in set
```

Q10. Lossless-join decomposition ensures:

Bloom's Level: BL2 – Understand | Marks: 5

✓ (a) No data is lost during decomposition — CORRECT ANSWER
(b) All FDs are preserved
(c) The relation is in BCNF
(d) No null values exist

Explanation:

A lossless-join (non-additive-join) decomposition guarantees that when the decomposed relations are joined back together (natural join), you get back exactly the original relation — no spurious rows and no lost rows/data. It does not by itself guarantee dependency preservation or any particular normal form.

```
Q10 Demo
mysql> SELECT * FROM Original;
+-----------+--------+------------+
| StudentID | Course | Instructor |
+-----------+--------+------------+
| 1         | DBMS   | Dr.Kumar   |
| 2         | DBMS   | Dr.Kumar   |
+-----------+--------+------------+
2 rows in set

mysql> SELECT R1.StudentID, R1.Course, R2.Instructor FROM R1 JOIN R2 ON R1.Course=R2.Course; -- rejoin repr
+-----------+--------+------------+
| StudentID | Course | Instructor |
+-----------+--------+------------+
| 1         | DBMS   | Dr.Kumar   |
| 2         | DBMS   | Dr.Kumar   |
+-----------+--------+------------+
2 rows in set
```

Answer Key Summary

Q.No	Correct Answer	Topic
Q1	(c) All attributes must be atomic	1NF – atomicity
Q2	(b) Has no partial dependencies on primary key	2NF – partial dependency
Q3	(d) 4NF	4NF – multi-valued dependency
Q4	(c) 3NF	BCNF vs 3NF
Q5	(b) X determines Y	Functional dependency notation
Q6	(d) Retrieval anomaly	Anomalies in unnormalized relations
Q7	(b) Any set of attributes that uniquely identifies tuples	Superkey definition
Q8	(c) Transitively dependent on A	Transitive dependency
Q9	(c) Project-Join Normal Form	5NF / PJNF
Q10	(a) No data is lost during decomposition	Lossless-join decomposition

---

Code of Conduct

I **K. Anil Kumar Reddy** (Name / Reg No: **192525239**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ______________________